

Module 14 — Frequently Asked Coding Questions

Take-home / live-coding staples for Spring Boot backend roles. Each item shows the idiomatic Spring solution and the traps interviewers look for. Keep controllers thin, use DTOs, validate input, handle errors centrally.

14.1 REST API Design (CRUD done right)

```
@RestController
@RequestMapping("/api/v1/orders")
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) { this.service = service; }

    @GetMapping("/{id}")
    OrderDto get(@PathVariable Long id) { return service.get(id); }

    @PostMapping
    ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrderReq req) {
        OrderDto dto = service.create(req);
        return ResponseEntity.created(URI.create("/api/v1/orders/" + dto.id())).body(dto);
    }

    @PutMapping("/{id}")
    OrderDto update(@PathVariable Long id, @Valid @RequestBody UpdateOrderReq req) {
        return service.update(id, req);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    void delete(@PathVariable Long id) { service.delete(id); }
}
```

Traps: correct status codes (201 + **Location**, 204 on delete), DTOs not entities, versioned path, idempotent PUT/DELETE.

14.2 Pagination

```
@GetMapping
Page<OrderDto> list(@RequestParam(defaultValue="0") int page,
                  @RequestParam(defaultValue="20") int size,
                  @RequestParam(defaultValue="createdAt,desc") String sort) {
    return service.list(PageRequest.of(page, Math.min(size, 100), parseSort(sort)));
}
```

Traps: cap page size; **Page** (with count) vs **Slice** (no count); **keyset/seek pagination** for deep pages (**WHERE id < :lastId ORDER BY id DESC LIMIT n**) since **OFFSET** degrades on large tables.

14.3 File Upload

```
@PostMapping(value="/files", consumes=MediaType.MULTIPART_FORM_DATA_VALUE)
FileDto upload(@RequestParam("file") MultipartFile file) throws IOException {
    if (file.isEmpty()) throw new BadRequestException("empty file");
    // validate size & content-type; stream to storage (S3) – don't hold in memory
    try (InputStream in = file.getInputStream()) { storage.put(key, in, file.getSize()); }
    return new FileDto(key, file.getOriginalFilename(), file.getSize());
}
```

Config: `spring.servlet.multipart.max-file-size`, `max-request-size`. **Traps:** validate size/type (magic bytes, not just extension), stream (don't load whole file into heap), sanitize filename, virus scan, store in object storage not the DB.

14.4 Exception Handling (global)

```
@RestControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(EntityNotFoundException.class)
    ProblemDetail notFound(EntityNotFoundException e) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getMessage());
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail invalid(MethodArgumentNotValidException e) {
        var pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setProperty("errors", e.getBindingResult().getFieldErrors().stream()
            .collect(toMap(FieldError::getField, f -> f.getDefaultMessage(), (a,b)->a)));
        return pd;
    }
    @ExceptionHandler(Exception.class)
    ProblemDetail generic(Exception e) {           // don't leak internals
        log.error("unhandled", e);
        return ProblemDetail.forStatus(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Traps: consistent `ProblemDetail` (RFC 7807); never leak stack traces to clients; map domain exceptions to codes; log with correlation id.

14.5 Validation

```
public record CreateOrderReq(
    @NotNull Long customerId,
    @NotEmpty List<@Valid LineItem> items,
    @Positive BigDecimal total) {}

public record LineItem(@NotBlank String sku, @Min(1) int qty) {}
```

Traps: `@Valid` cascades into nested/collection elements; use groups for create vs update; custom `ConstraintValidator` for cross-field rules; `@Validated` on `@Service` for method-param validation.

14.6 Security (secured endpoint)

```
@PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
@GetMapping("/users/{id}")
UserDto get(@PathVariable Long id) { ... }
```

Traps: method + URL security; principal-based checks (owner access); never trust client-supplied identity; validate JWT (Module 6).

14.7 Transactions

```
@Transactional
public OrderDto placeOrder(CreateOrderReq req) {
    Order o = orderRepo.save(Order.from(req));
    inventory.reserve(o);           // same tx (local); external? use saga/outbox
    return OrderDto.from(o);
}
```

Traps: self-invocation bypasses proxy; checked exceptions don't roll back by default (`rollbackFor`); don't call remote APIs inside a tx (pool exhaustion); `REQUIRES_NEW` for independent units; keep transactions short.

14.8 Caching

```
@Cacheable(value="user", key="#id", sync=true) // sync -> stampede protection
public UserDto get(Long id) { return repo.findById(id).map(UserDto::from).orElseThrow(); }

@CacheEvict(value="user", key="#dto.id()")
public UserDto update(UserDto dto) { ... }
```

Traps: evict on write (not update-in-place); TTL via cache config; `sync=true` to prevent stampede; don't cache per-request/user-specific data under shared keys; `@CachePut` vs `@Cacheable` semantics.

14.9 Retry Logic

```
@Retryable(retryFor = TransientException.class,
           maxAttempts = 4,
           backoff = @Backoff(delay = 200, multiplier = 2, random = true)) // jitter
public Price fetch(String sku) { return client.get(sku); }

@Recover
Price recover(TransientException e, String sku) { return cache.last(sku); }
```

Or Resilience4j `@Retry` + `@CircuitBreaker` (Module 7). **Traps:** only retry **idempotent** ops; exponential backoff + jitter; cap attempts; combine with circuit breaker + timeout; don't retry 4xx.

14.10 Async Processing

```
@Async("taskExecutor")
public CompletableFuture<Report> generate(Long id) {
    return CompletableFuture.completedFuture(build(id));
}

@Bean
ThreadPoolTaskExecutor taskExecutor() {
    var ex = new ThreadPoolTaskExecutor();
    ex.setCorePoolSize(8); ex.setMaxPoolSize(16);
    ex.setQueueCapacity(500); ex.setThreadNamePrefix("async-");
    ex.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
    return ex;
}
```

Traps: `@Async` needs `@EnableAsync` + a proxy call (self-invocation bypass!); provide a **dedicated bounded executor** (don't use the default `SimpleAsyncTaskExecutor` which makes unbounded threads); return `CompletableFuture/void`; exceptions in `void @Async` are swallowed (use `AsyncUncaughtExceptionHandler`).

Module 14 — One-Page Cheat Sheet

Task	Do	Avoid
REST	DTOs, status codes, <code>ResponseEntity</code>	exposing entities
Pagination	cap size; keyset for deep	huge OFFSET
Upload	stream, validate type/size	load into heap
Errors	<code>@RestControllerAdvice</code> + <code>ProblemDetail</code>	leaking stack traces
Validation	<code>@Valid</code> , nested, groups	validating entities
Security	method + URL, owner checks	trusting client id
Tx	short, <code>rollbackFor</code> , no remote calls	self-invocation, long tx
Cache	evict on write, TTL, <code>sync</code>	shared key for user data
Retry	idempotent, backoff+jitter, cap	retrying 4xx / non-idempotent
Async	<code>@EnableAsync</code> , bounded executor	self-invoke, unbounded pool

Module 14 — Top Interview Questions

1. Design a RESTful CRUD API (status codes, DTOs, versioning).
2. Offset vs keyset pagination — when and why.
3. Safely handle large file uploads.
4. Centralized error handling with consistent schema.
5. Cascade validation into nested objects; cross-field rules.
6. Secure an endpoint for owner-or-admin access.
7. Why does `@Transactional`/`@Async`/`@Cacheable` fail on self-invocation?
8. Implement retry with backoff for a flaky dependency.
9. Cache invalidation strategy for updates.
10. Configure an async executor correctly.

Module 14 — Common Mistakes

- Exposing JPA entities directly (serialization + lazy issues).
- Self-invocation breaking proxy-based annotations.
- Unbounded async executor / retries.
- Retrying non-idempotent operations.
- Leaking internal errors to clients.

Module 14 — Mock Interview

1. “Add pagination to a 100M-row table endpoint.” → keyset/seek pagination + cap size; avoid deep OFFSET.
2. “`@Async` method runs synchronously.” → missing `@EnableAsync` or self-invocation; call through an injected bean.
3. “Retries are hammering a failing service.” → add backoff+jitter, cap attempts, circuit breaker; only retry idempotent/transient.
4. “Standardize errors across the API.” → `@RestControllerAdvice` + `ProblemDetail`, map domain exceptions, hide internals.
5. “Upload endpoint OOMs on big files.” → stream to object storage, don't buffer in heap; enforce max size.

Next → Module 15: Company Interview Questions.